**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

# LESSON 311 · Unfamiliar programming challenges

Name: _____ Class: _____ Date: _____

Time: _____ Teacher: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

## Do Now · Recall previous learning

Q1. What is a loop invariant?

Q2. What distinguishes a run from a distinct value?

Q3. Why handle empty arrays explicitly?

Q4. For a=[1,1,2,1], give count of distinct values and runs.

BLACK PEN Answer all questions first. GREEN PEN Then: tick correct answers; rewrite mistakes.

# I Do · Read new knowledge

## Key vocabulary

run — Consecutive equal-value group.

invariant — Property maintained across iterations.

edge case — Boundary or unusual input.

distinct — Different values irrespective of position.

## New learning

Understand — A run is a maximal group of equal consecutive values, not a unique value across the whole array.

Apply — Maintain invariant: after index i, the count equals the number of runs in the processed prefix.

Evaluate — Handle empty input and test all-equal, alternating and separated-repeat arrays.

## Worked example

Trace CountRuns on [4,4,5,5,5].

1. Predict first — Consecutive repeated fives belong to the same second run.

2. Explain and check — Explain the reason for the result.

# We Do · Practise with guidance

Trace CountRuns on [1,2,1,2].

```
Each neighbouring pair differs; return4.
```

Predict first

There are four runs but only two distinct values.

Explain and check

Explain the reason for the result.

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

# You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

### Question 1

Trace CountRuns for [3,3,4,4,3,5,5]. State where runs increment. [6]

### Question 2

Write a valid C# CountRuns(int[] a) returning runs and handling empty input. [8]

### Question 3

For [1,2,1], compare runs and distinct counts, and assess time/extra-space complexity of CountRuns. [6]

# Exit Ticket · Answer before checking

Runs always equal number of unique values.

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

# Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

### Page 1 onwards · Do Now

Q1. A property true before and after each completed iteration.

Q2. Runs count contiguous groups; distinct counts globally unique values.

Q3. Otherwise a[0] could be invalid and initial count wrong.

Q4. Distinct values 2; runs 3 ([1,1],[2],[1]).

### You Do pages · Independent questions

Q1. Start runs1 [1]; change to4 makes2 [2]; change to3 makes3 [1]; change to5 makes4 [1]; return4 [1].

Q2. Mark scheme [8]: an int-returning method accepting int[] [1]; detect an empty array and return 0 [2]; initialise the run count to 1 [1]; loop from index 1 [1]; compare neighbouring values [1]; increment on change [1]; return the result [1]. Equivalent correct C# earns the same marks.
CODE:

```
static int CountRuns(int[] a)
{
if (a.Length == 0) return 0;
int runs = 1;
for (int i = 1; i < a.Length; i++)
{
if (a[i] != a[i - 1]) runs++;
}
return runs;
}
```

Q3. Runs3 [1], distinct2 [1]; adjacency explains difference [1]; O(n) time [2]; O(1) auxiliary storage [1].

### Exit Ticket · Model answer

False. A value can occur in more than one run.

# Try the question again

**BLACK PEN** Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Choose one question you got wrong. All correct? Choose the hardest.

### 1. Copy the whole original question:

### 2. Write a complete new answer in GREEN PEN. Show working if needed: